Python Zero to Projects

Chapter 7: Sets

Structured Study Notes | python-zero-to-projects

Bridge from Chapter 6 (Tuples)

In Chapter 6 you learned about tuples: ordered, immutable sequences that allow duplicate values and support indexing and slicing. Sets are the next step - they share some surface syntax (curly braces) but behave very differently:

Property	Tuple (Ch. 6)	Set (Ch. 7)
Ordered?	Yes	No
Mutable?	No (immutable)	Yes (mostly)
Duplicates?	Yes	No - auto-removed
Indexing?	Yes	No
Slicing?	Yes	No

1. Creating Sets and Core Properties

A set is a collection of **unique, unordered** elements. There are two ways to create one:

1a. Creating a set with curly braces {}

```
# Using curly braces - works when you have at least one element
fruits = {"apple", "banana", "cherry", "apple"}   # "apple" appears twice
print(fruits)   # Output: {'banana', 'cherry', 'apple'}  (duplicates removed)
print(type(fruits))   # <class 'set'>
```

1b. Creating a set with set()

```
# Using set() - required for an empty set!
empty_set = set()          # Correct: creates an empty set
wrong     = {}             # This creates an empty DICT, not a set!

# set() can also convert other iterables
nums = set([1, 2, 3, 2, 1])   # Converts list -> set
```

```
print(nums)   # {1, 2, 3}  (duplicates removed automatically)
```

1c. Automatic Removal of Duplicates

One of the most powerful features of sets is that they **cannot hold duplicate values**. When you create a set with repeated elements, Python automatically keeps only one copy of each:

```
scores = {95, 80, 95, 70, 80, 100}
print(scores)    # {80, 95, 100, 70} - each value appears once
                 # Note: the output order may vary (sets are unordered)
```

1d. Sets Do NOT Support Indexing or Slicing

This is a critical difference from strings (Ch. 4), lists (Ch. 5), and tuples (Ch. 6). Because sets are **unordered**, there is no "first" or "second" element, so indexing and slicing make no sense and will raise a TypeError:

```
# Strings, Lists, Tuples - indexing WORKS:
my_str   = "hello"
my_list  = [10, 20, 30]
my_tuple = (10, 20, 30)
print(my_str[0])     # "h"  - OK
print(my_list[1])    # 20   - OK
print(my_tuple[2])   # 30   - OK

# Sets - indexing raises an ERROR:
my_set = {10, 20, 30}
print(my_set[0])     # TypeError: 'set' object is not subscriptable
```

1e. len() and Membership Operators (in / not in)

You already used len() and in/not in with strings, lists, and tuples. They work the same way with sets - but membership testing is **significantly faster** with sets for large collections because sets use a hash table internally (O(1) vs O(n) for lists/tuples):

```
my_set = {"apple", "banana", "cherry"}

# len() - same as before
print(len(my_set))       # 3

# in / not in - same syntax, much faster performance
print("banana" in my_set)      # True
print("grape" in my_set)       # False
print("grape" not in my_set)   # True

# Practical advantage: checking if an item is in a set of 1 million
# elements is nearly instantaneous - much faster than scanning a list
```

NOTE: The fast membership testing is why sets are often the right choice when you need to answer "is X in this collection?" repeatedly on large data.

2. Set Methods

Sets have built-in methods for adding and removing elements. Unlike lists, there is no "insert at position" because sets have no order.

2a. add() - Add a Single Element

```
colors = {"red", "blue"}
colors.add("green")
print(colors)    # {"red", "blue", "green"}  (order may vary)

# Adding a duplicate has no effect
colors.add("red")
print(colors)    # Still {"red", "blue", "green"}
```

2b. update() - Add Multiple Elements at Once

update() accepts any iterable (list, tuple, another set, etc.) and adds all its elements to the set:

```
nums = {1, 2, 3}
nums.update([4, 5, 6])     # Add from a list
print(nums)                # {1, 2, 3, 4, 5, 6}

nums.update({7, 8}, [9])   # Can pass multiple iterables
print(nums)                # {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

2c. remove() - Remove a Specific Element (error if missing)

```
fruits = {"apple", "banana", "cherry"}
fruits.remove("banana")
print(fruits)    # {"apple", "cherry"}

# Trying to remove an element that does not exist raises KeyError
fruits.remove("mango")    # KeyError: 'mango'
```

2d. discard() - Remove Safely (NO error if missing)

Key difference from remove(): discard() does nothing if the element is not found, instead of raising an error. Use discard() when you are not sure if the element exists:

```
fruits = {"apple", "banana", "cherry"}
fruits.discard("banana")   # Removes "banana" - same as remove()
print(fruits)              # {"apple", "cherry"}

# The big difference: no error if element is missing
fruits.discard("mango")    # Does nothing - no KeyError raised
print(fruits)              # {"apple", "cherry"} - unchanged
```

2e. pop() - Remove and Return an Arbitrary Element

Because sets are unordered, you cannot control which element pop() removes - it removes and returns an arbitrary element. Raises KeyError if the set is empty:

```
my_set = {10, 20, 30, 40}
removed = my_set.pop()     # Removes and returns some element
print(removed)             # Could be 10, 20, 30, or 40 - unpredictable
print(my_set)              # The remaining 3 elements
```

2f. clear() - Remove ALL Elements

```
my_set = {1, 2, 3, 4, 5}
my_set.clear()
print(my_set)    # set()   (empty set - NOT {})
print(len(my_set))   # 0
```

3. Set Operations

Sets support four classic mathematical operations. Each has both an operator symbol and a method form - both produce the same result:

> **IMPORTANT NOTE on Symbols: The symbols &, |, and ^ were introduced in Chapter 3 as bitwise operators. When used on integers, they operate at the binary bit level (e.g., 5 & 3 = 1). When used on sets, they perform set operations instead. Python knows which behavior to use based on the type of the operands.**

```
# Setup for all examples below
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
```

3a. Union (| or .union()) - All Elements from Both Sets

Returns every element that appears in A, B, or both:

```
result = A | B
print(result)        # {1, 2, 3, 4, 5, 6, 7, 8}

# Equivalent using the method form:
result = A.union(B)
print(result)        # {1, 2, 3, 4, 5, 6, 7, 8}
```

3b. Intersection (& or .intersection()) - Common Elements Only

Returns only elements that appear in BOTH A and B:

```
result = A & B
print(result)        # {4, 5}

# Equivalent using the method form:
result = A.intersection(B)
print(result)        # {4, 5}
```

3c. Difference (- or .difference()) - In A but NOT in B

Returns elements in A that do not appear in B:

```
result = A - B
print(result)        # {1, 2, 3}

# Equivalent using the method form:
result = A.difference(B)
print(result)        # {1, 2, 3}

# Note: A - B is different from B - A
result = B - A
print(result)        # {6, 7, 8}  (in B but not in A)
```

3d. Symmetric Difference (^ or .symmetric_difference()) - In One But Not Both

Returns elements that are in A or B, but NOT in both:

```
result = A ^ B
print(result)        # {1, 2, 3, 6, 7, 8}  (excludes the common 4 and 5)

# Equivalent using the method form:
result = A.symmetric_difference(B)
print(result)        # {1, 2, 3, 6, 7, 8}
```

Venn Diagrams of Set Operations

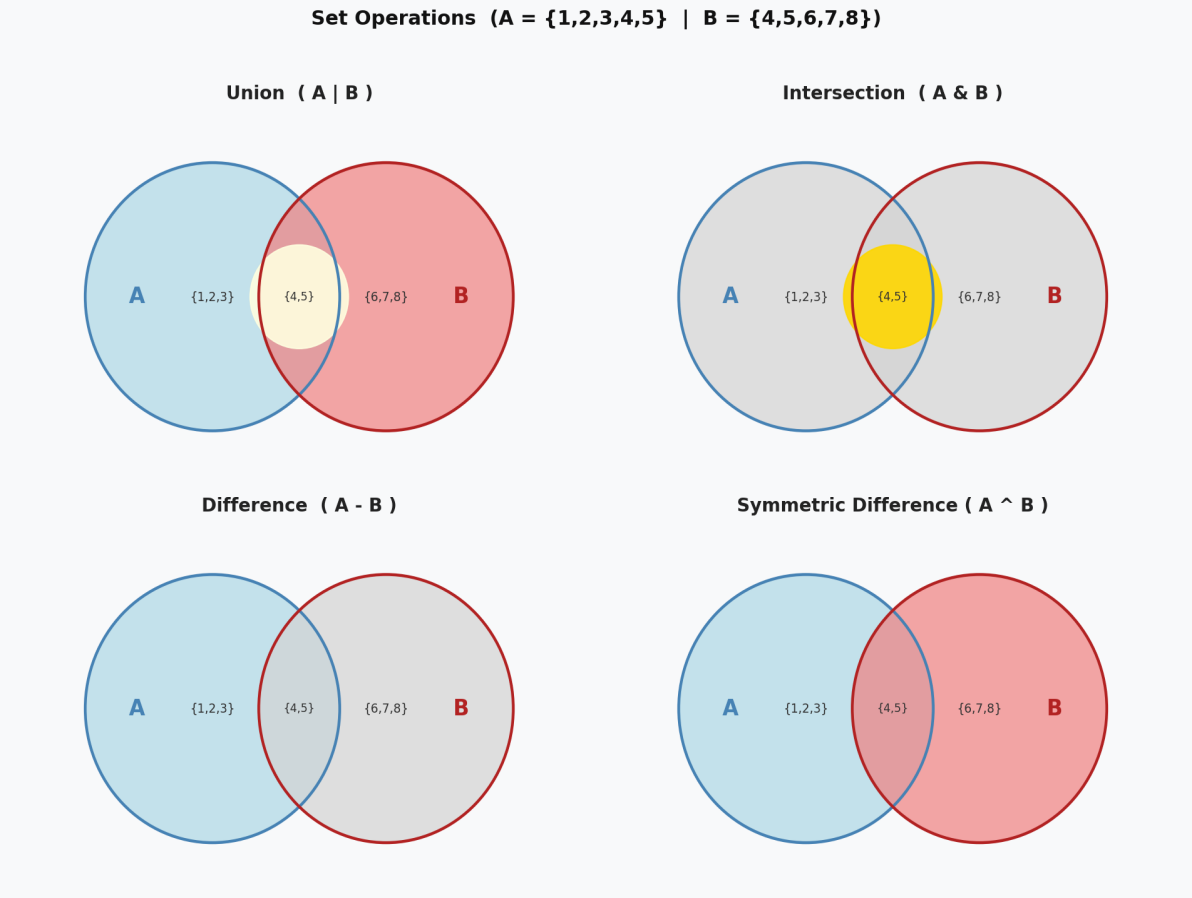

Figure 1: Venn diagrams illustrating Union, Intersection, Difference, and Symmetric Difference for A = {1,2,3,4,5} and B = {4,5,6,7,8}. Highlighted regions show the result of each operation.

4. Frozen Sets

A **frozenset** is an immutable version of a set. Once created, you cannot add, remove, or modify its elements. Think of it as a "frozen snapshot" of a set.

Why do frozen sets exist?

Python requires dictionary keys and set elements to be **hashable** - meaning their value must never change after creation. Regular sets are mutable (they can change), so they are not hashable and cannot be used as dictionary keys or placed inside another set. Frozen sets ARE hashable, making them useful in these situations:

```
# Creating a frozenset
fs = frozenset({1, 2, 3, 4, 5})
print(fs)          # frozenset({1, 2, 3, 4, 5})
print(type(fs))    # <class 'frozenset'>

# frozensets support all READ operations
print(len(fs))           # 5
print(3 in fs)           # True
print(fs & {2, 3, 9})   # frozenset({2, 3})  - intersection still works

# frozensets DO NOT support modification
fs.add(6)          # AttributeError: 'frozenset' has no attribute 'add'
fs.remove(1)       # AttributeError: 'frozenset' has no attribute 'remove'
```

Using frozenset as a dictionary key (not possible with regular set):

```
# Regular set as dict key -> TypeError
my_dict = {}
my_set = {1, 2, 3}
# my_dict[my_set] = "value"   # TypeError: unhashable type: 'set'

# Frozenset as dict key -> Works!
my_frozenset = frozenset({1, 2, 3})
my_dict[my_frozenset] = "works fine"
print(my_dict)     # {frozenset({1, 2, 3}): 'works fine'}
```

5. Converting Between Collections

A very common Python pattern is to convert a list with duplicates into a set to remove duplicates, then convert back to a list if you need indexing or a guaranteed order again.

Practical Example: Removing Duplicates from a List

```
# Step 1: Start with a list that has duplicate values
raw_scores = [95, 80, 70, 95, 80, 100, 70, 60]
```

```
print("Original list:", raw_scores)
print("Length:", len(raw_scores))   # 8 elements

# Step 2: Convert to a set - duplicates are removed automatically
unique_scores = set(raw_scores)
print("As set:", unique_scores)     # {60, 70, 80, 95, 100} - 5 unique values

# Step 3: Convert back to a list if you need indexing again
clean_list = list(unique_scores)
print("Clean list:", clean_list)

# Step 4: Sort if you need a specific order
sorted_list = sorted(clean_list)
print("Sorted:", sorted_list)   # [60, 70, 80, 95, 100]
```

CAUTION: When converting list -> set -> list, the original order of elements is NOT preserved (because sets are unordered). If order matters, sort() the result afterwards, or use a different approach like a loop with an "already seen" check.

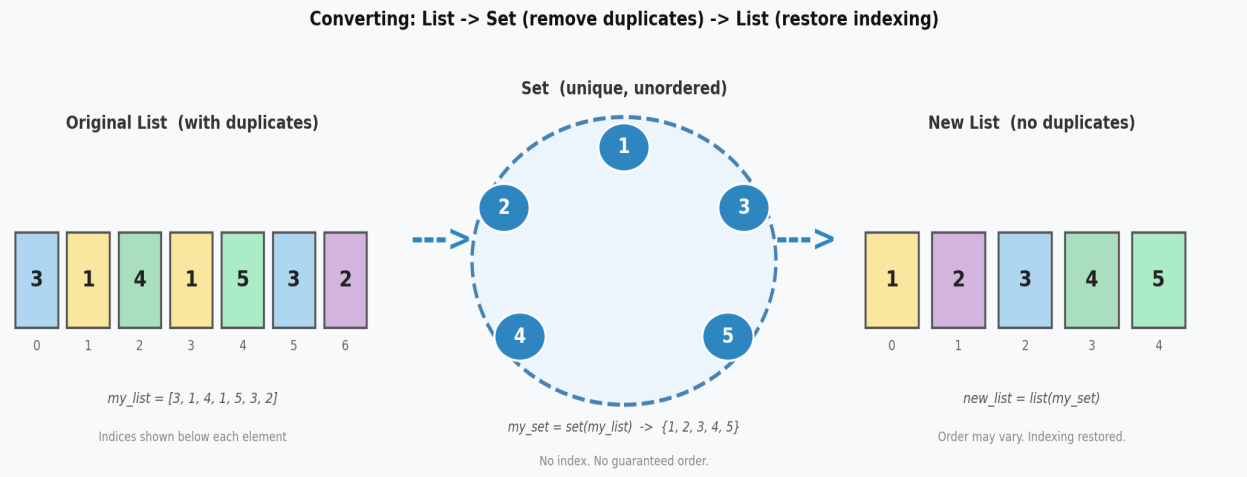

Figure 2: Converting a list with duplicates into a set removes duplicates, then converting back to a list restores indexing. Note that order may change.

6. Set Comprehension

Just like list comprehension was introduced in Chapter 5 as a shorthand for creating lists, **set comprehension** lets you create sets using a similar compact syntax. You will understand this more deeply after Chapter 10 (Loops), but it is useful to recognize the pattern now.

Basic Syntax:

```
# General form:
# { expression  for  item  in  iterable }

# Compare: list comprehension uses square brackets []
# Set comprehension uses curly braces {}
# List comprehension  ->  returns a list
```

```
squares_list = [x * x for x in range(1, 6)]
print(squares_list)   # [1, 4, 9, 16, 25]  (a list, ordered, duplicates OK)

# Set comprehension  ->  returns a set
squares_set = {x * x for x in range(1, 6)}
print(squares_set)    # {1, 4, 9, 16, 25}  (a set, unordered, unique)
```

Set Comprehension Automatically Removes Duplicates:

```
# Even if the expression produces duplicates, the set keeps only unique values
nums = [-3, -2, -1, 0, 1, 2, 3]

# Squaring negatives and positives gives the same result
squares = {x * x for x in nums}
print(squares)   # {0, 1, 4, 9}  (not 7 elements - duplicates removed)
```

PREVIEW: Chapter 10 on Loops will cover for-loop syntax in detail. For now, just recognize the pattern: the expression on the left transforms each item, and for item in iterable cycles through all values.

7. Quick Comparison: List vs Tuple vs Set

This table ties together Chapters 5, 6, and 7 into one reference:

Property	List (Ch. 5)	Tuple (Ch. 6)	Set (Ch. 7)
Syntax	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}
Ordered?	Yes	Yes	No
Mutable?	Yes	No (immutable)	Yes
Duplicates?	Yes	Yes	No
Indexing?	Yes (list[0])	Yes (tup[0])	No -> TypeError
Slicing?	Yes (list[1:3])	Yes (tup[1:3])	No -> TypeError
Typical Use	Ordered mutable sequences	Fixed data, return values	Unique items, membership tests, set math
Membership test	in / not in (slow O(n))	in / not in (slow O(n))	in / not in (fast O(1))

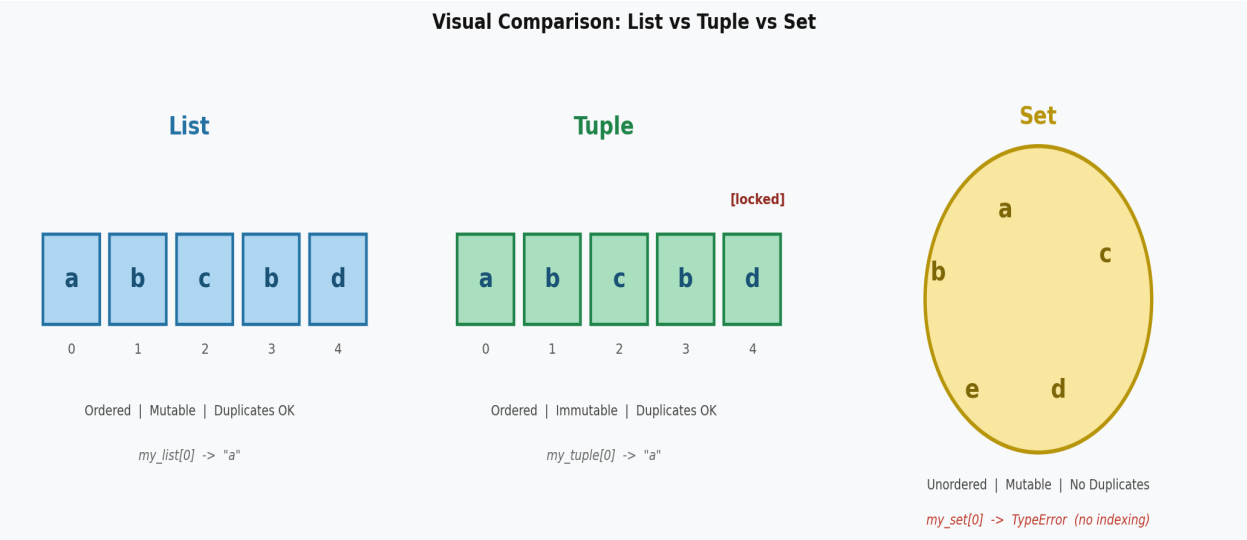

Figure 3: Visual comparison showing list and tuple as ordered, indexed boxes vs set as an unordered bag of unique elements.

Common Beginner Mistakes with Sets

Mistake 1: {} creates an empty DICT, not an empty set

```
# WRONG: This creates an empty dictionary, not a set!
empty = {}
print(type(empty))    # <class 'dict'>  <- not what you wanted!

# CORRECT: Use set() to create an empty set
empty = set()
print(type(empty))    # <class 'set'>   <- correct
```

Mistake 2: Trying to access a set element by index

```
my_set = {10, 20, 30}
print(my_set[0])    # TypeError: 'set' object is not subscriptable

# Fix: if you need indexing, convert to list first
my_list = list(my_set)
print(my_list[0])   # Works (but the order may surprise you)
```

Mistake 3: remove() raises KeyError, discard() does not

```
my_set = {"apple", "banana"}

# DANGEROUS if you are not sure the element exists:
my_set.remove("mango")    # KeyError: 'mango'

# SAFE alternative:
my_set.discard("mango")   # No error - does nothing if missing
```

Mistake 4: Adding an unhashable type (like a list) into a set

```
# Sets can only contain hashable (unchangeable) objects
my_set = {1, 2, 3}

# Lists are mutable -> not hashable -> cannot go inside a set
my_set.add([4, 5])    # TypeError: unhashable type: 'list'

# Fix: convert to a tuple (immutable -> hashable)
my_set.add((4, 5))    # Works! Tuples are hashable.
```

Mistake 5: Confusing set operators (&, |, ^) with bitwise operators from Ch. 3

```
# Chapter 3 bitwise operations on INTEGERS:
print(5 & 3)    # 1  (binary AND: 101 & 011 = 001)
print(5 | 3)    # 7  (binary OR:  101 | 011 = 111)
print(5 ^ 3)    # 6  (binary XOR: 101 ^ 011 = 110)

# Same symbols on SETS perform set operations:
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
print(A & B)    # {4, 5}             (intersection)
print(A | B)    # {1,2,3,4,5,6,7,8}  (union)
print(A ^ B)    # {1,2,3,6,7,8}      (symmetric difference)

# Python uses the TYPE of the operands to decide which behavior to use.
```

Preview: Chapter 8 - Dictionaries

Chapter 8 introduces dictionaries, which also use curly braces {} like sets. The key difference is that dictionaries store **key-value pairs** instead of single values:

```
# Set: single values inside {}
my_set  = {"apple", "banana", "cherry"}

# Dictionary: key-value PAIRS inside {}, separated by colon :
my_dict = {"name": "Alice", "age": 30, "city": "Jaipur"}
          #  key     value     key    value    key     value

# Access by key (not by index!)
print(my_dict["name"])   # "Alice"
```

Just like set elements must be unique, **dictionary keys must also be unique**. If you add a key that already exists, the old value is overwritten. Dictionary values, however, can repeat freely.

Chapter 7 Summary and Recap

- Sets are unordered collections of UNIQUE elements. Duplicates are removed automatically.

- Create with {} (non-empty) or set() (always works, including empty sets).
- Sets do NOT support indexing or slicing. This is a key break from lists and tuples.
- len() counts elements; in/not in tests membership (faster than lists for large sets).
- add() adds one element; update() adds many; remove() removes and raises KeyError if missing; discard() removes safely (no error); pop() removes arbitrary element; clear() empties the set.
- Set operations: | (union), & (intersection), - (difference), ^ (symmetric difference). Each has an equivalent method form.
- frozenset() creates an immutable set - useful as dict keys or inside other sets.
- Convert list -> set to remove duplicates, then back to list if indexing is needed. Order is not preserved.
- Set comprehension syntax: { expression for item in iterable }.
- Common pitfalls: {} is an empty dict not a set; no indexing on sets; remove() vs discard(); only hashable types inside sets; &/|/^ behave differently on ints vs sets.
- Chapter 8 preview: dicts also use {} but hold key-value pairs; keys must be unique.

--- End of Chapter 7 Notes ---